Concordia University

Department of Computer Science and Software Engineering

SOEN 345 Project Report

Cloud-based Ticket Reservation Application

Student Name:	Harjot Minhas
Student ID:	40315397
Course:	SOEN 345 – Software Testing, Verification and Quality Assurance
Term:	Winter 2026
Submission Date:	April 13, 2026
Public GitHub Repository:	https://github.com/M1nha5/soen345-ticket-reservation

Final Submission

Contents

1 Project Overview **2**
1.1 Project Objectives 2

2 Software Development Method **2**
2.1 Adopted Methodology 2
2.2 Lifecycle and Iterations 2
2.3 Definition of Done 3

3 Requirements Analysis **3**
3.1 Functional Requirements Coverage 3
3.2 Non-Functional Requirements Coverage 4

4 System Architecture and Design **4**
4.1 Architecture Overview 4
4.2 Use Case Design 4
4.3 UML Diagrams 6
4.4 Database Design (Firestore) 8
4.5 Core Sequence (Reservation) 8

5 Implementation Summary **8**
5.1 Role-Based Behavior 8
5.2 Security and Data Integrity 8
5.3 User Interface 8

6 Software Testing Method and Results **9**
6.1 Testing Strategy 9
6.2 Unit and Component Tests 9
6.3 Functional and Acceptance Tests 9

7 Development Tools and Collaboration **10**

8 Application Screens and Test Evidence **11**
8.1 Core Application Screens 11
8.2 Testing and Notification Evidence 22
8.3 Additional Evidence Files 24

9 Conclusion **25**

1 Project Overview

This project implements a cloud-based Android ticket reservation application where users can browse events, reserve tickets, cancel reservations, and receive confirmation records. The system supports multiple roles:

- Customer
- Organizer
- Admin

The implementation follows the course requirement to use Java and demonstrates software engineering, testing, and quality assurance practices with a Firebase cloud backend.

1.1 Project Objectives

- Build a mobile ticket reservation solution using Java on Android.
- Implement role-based workflows for customer, organizer, and admin.
- Provide cloud-based storage and authentication.
- Validate correctness through unit, component, functional, and acceptance testing.
- Use professional engineering tools: GitHub, CI/CD, Android Studio, JUnit, and Firebase.

2 Software Development Method

2.1 Adopted Methodology

The project used an iterative Scrum-style development process. Work was delivered in short increments with frequent integration and testing.

2.2 Lifecycle and Iterations

Iteration 1: Foundation

- Android project setup
- Firebase integration (Auth, Firestore, Functions)
- Core data models and service layer

Iteration 2: Core Functional Features

- Email/password and phone OTP authentication
- Event listing, filtering, reservation and cancellation
- Organizer registration and admin approval flow
- Event management (add/edit/cancel)

Iteration 3: Hardening and Quality

- Firestore rules hardening

- Confirmation function reliability improvements
- UI improvements and empty-state handling
- CI stability improvements and regression testing
- Final release build signing and deployment readiness

2.3 Definition of Done

A work item was considered done only when:

- implementation was complete in app/backend,
- feature behavior was validated by tests or acceptance checks,
- integration worked with Firebase,
- documentation was updated in `docs/`.

3 Requirements Analysis

3.1 Functional Requirements Coverage

ID	Requirement	Status	Implementation Evidence
FR1	User registration with email or phone number	Implemented	Email registration and phone OTP login screens
FR2	View list of available events	Implemented	Event list screen and Firestore `events` collection
FR3	Search/filter events by date, location, category	Implemented	In-app filter logic and UI controls
FR4	Cancel reservations	Implemented	Reservation list with cancel action and transaction logic
FR5	Receive confirmation via email/SMS channel record	Implemented	Callable Cloud Function writes confirmation outcome
FR6	Admin/Organizer add new event	Implemented	Event editor and role-based access
FR7	Admin/Organizer edit existing event	Implemented	Event editor update flow

FR8	Admin/Organizer cancel event	Implemented	Event status transition to cancelled

3.2 Non-Functional Requirements Coverage

ID	Requirement	Status	Implementation Strategy
NFR1	Support concurrent users without performance degradation	Implemented	Firestore transactions for reservation/cancellation preserve inventory consistency under concurrent access
NFR2	Cloud-based high availability	Implemented	Firebase Auth, Firestore, and Cloud Functions managed cloud infrastructure
NFR3	UI should be simple and user-friendly	Implemented	Role-based dashboard, improved visual hierarchy, clearer forms, and explicit empty states

4 System Architecture and Design

4.1 Architecture Overview

The application follows a client-cloud architecture:

- Android app (presentation + client logic)
- Firebase Authentication (identity)
- Cloud Firestore (data persistence)
- Firebase Cloud Functions (server-side confirmation processing)

Architecture interactions are shown in the UML diagrams below and follow the same flow: Android app communicates with Firebase Authentication, Cloud Firestore, and Cloud Functions, while Cloud Functions also communicates with Firestore.

4.2 Use Case Design

Primary actors:

- Customer
- Organizer
- Admin

Customer use cases:

- Register/Login
- Browse events
- Search/filter events
- Reserve tickets
- Cancel reservation

Organizer/Admin use cases:

- Create event
- Edit event
- Cancel event

Admin-only use cases:

- Approve/reject organizer accounts

4.3 UML Diagrams

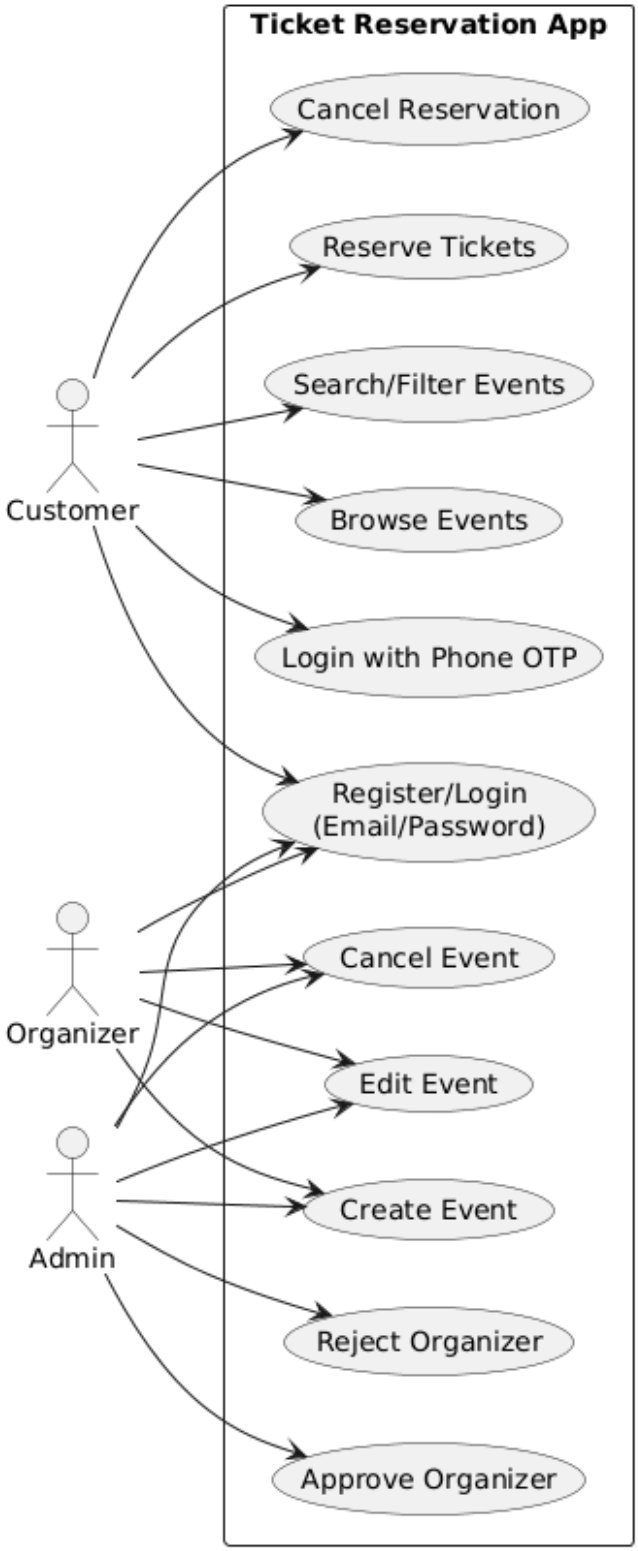

Figure 1: Use case diagram.

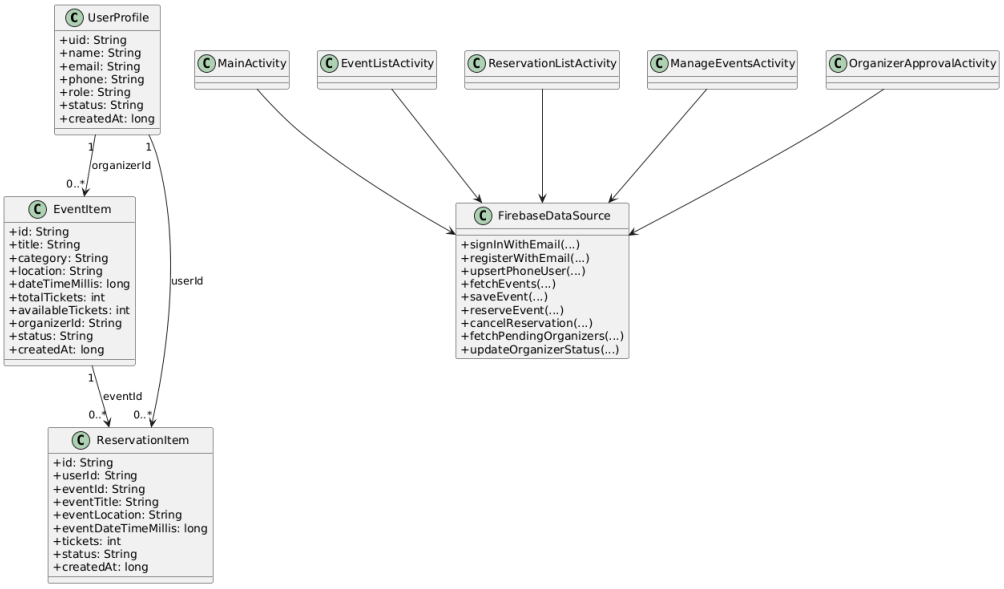

Figure 2: Class diagram.

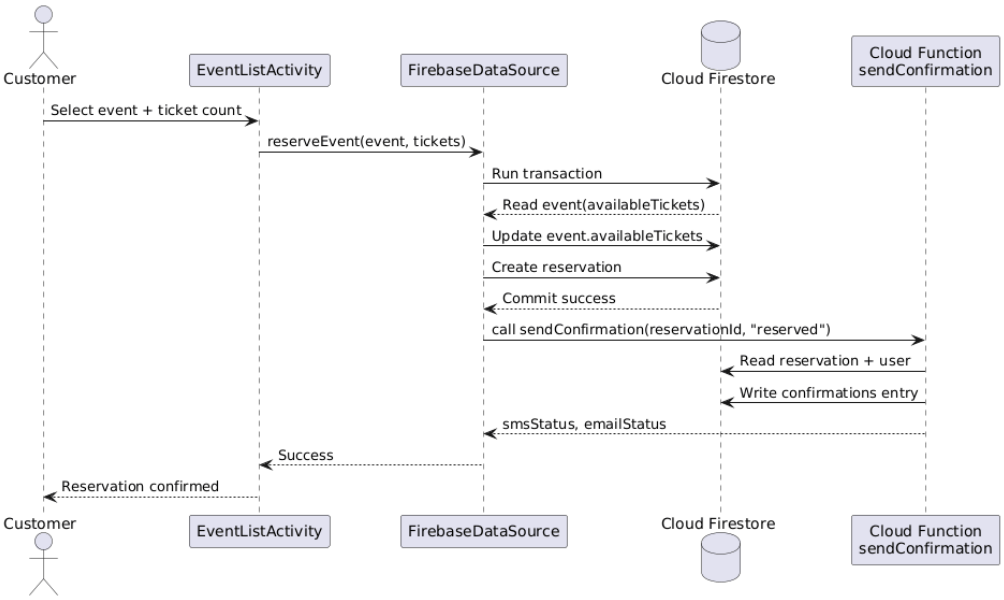

Figure 3: Sequence diagram for reservation flow.

4.4 Database Design (Firestore)

Collection: `users`

- `uid`, `name`, `email`, `phone`, `role`, `status`, `createdAt`

Collection: `events`

- `id`, `title`, `category`, `location`, `dateTimeMillis`
- `totalTickets`, `availableTickets`, `organizerId`, `status`, `createdAt`

Collection: `reservations`

- `id`, `userId`, `eventId`, `eventTitle`, `eventLocation`
- `eventDateTimeMillis`, `tickets`, `status`, `createdAt`

Collection: `confirmations`

- `reservationId`, `userId`, `eventId`, `action`
- `smsStatus`, `emailStatus`, `createdAt`

4.5 Core Sequence (Reservation)

1. Customer selects an event and ticket quantity.
2. Android app requests reservation through service layer.
3. Firestore transaction loads latest event document.
4. System validates availability and decrements available tickets atomically.
5. Reservation document is created in the same transaction.
6. App calls `sendConfirmation` Cloud Function.
7. Cloud Function writes confirmation outcome to `confirmations`.

5 Implementation Summary

5.1 Role-Based Behavior

- **Customer:** browse/filter events, reserve/cancel tickets, view reservations.
- **Organizer:** create/edit/cancel events only after admin approval.
- **Admin:** approve/reject organizers and manage events.

5.2 Security and Data Integrity

- Firestore rules enforce role-based authorization.
- Rules prevent self role/status privilege escalation.
- Reservation/event updates use constrained write rules and transaction checks.
- Cloud Function callable endpoint verifies authenticated caller and permissions.

5.3 User Interface

- Redesigned, user-friendly screens for authentication, dashboards, and event lists.

- Clear empty states for no events and no reservations.
- Improved reservation input using number picker for ticket count.
- Phone input normalization for North American numbers (auto `+1` support).

6 Software Testing Method and Results

6.1 Testing Strategy

Testing was performed across four levels:

- Unit tests for policy/filter logic
- Component tests for booking flow logic
- Functional UI checks using Espresso
- Manual acceptance tests for complete end-to-end scenarios

6.2 Unit and Component Tests

Execution command:

```
./gradlew testDebugUnitTest
```

Result summary (April 13, 2026):

- Total: 10
- Passed: 10
- Failed: 0
- Errors: 0
- Skipped: 0

Covered test suites:

- Event filtering logic
- Role policy and organizer approval constraints
- Reservation policy validation
- Booking component flow consistency

6.3 Functional and Acceptance Tests

Functional UI test:

- Espresso test verifies login screen actions and visibility.

Acceptance scenarios executed:

- Customer registration and login (email/password)
- Organizer registration (pending status)
- Admin approval/rejection for organizers
- Event creation/update/cancellation by authorized role
- Customer browsing/filtering/reserving/cancelling

- Phone OTP authentication flow
- Confirmation generation on reservation state changes

Artifacts:

- `docs/testing-plan.md`
- `docs/acceptance-results.md`

7 Development Tools and Collaboration

Category	Tooling Used
IDE	Android Studio
Programming Language	Java (Android app), JavaScript (Cloud Functions)
Version Control	Git + GitHub
CI/CD	GitHub Actions (`.github/workflows/android-ci.yml`)
Testing Frameworks	JUnit, Mockito, Espresso
Cloud Platform	Firebase Authentication, Firestore, Cloud Functions

The repository was continuously updated on the `main` branch with feature-based commits and verification passes after major changes.

8 Application Screens and Test Evidence

8.1 Core Application Screens

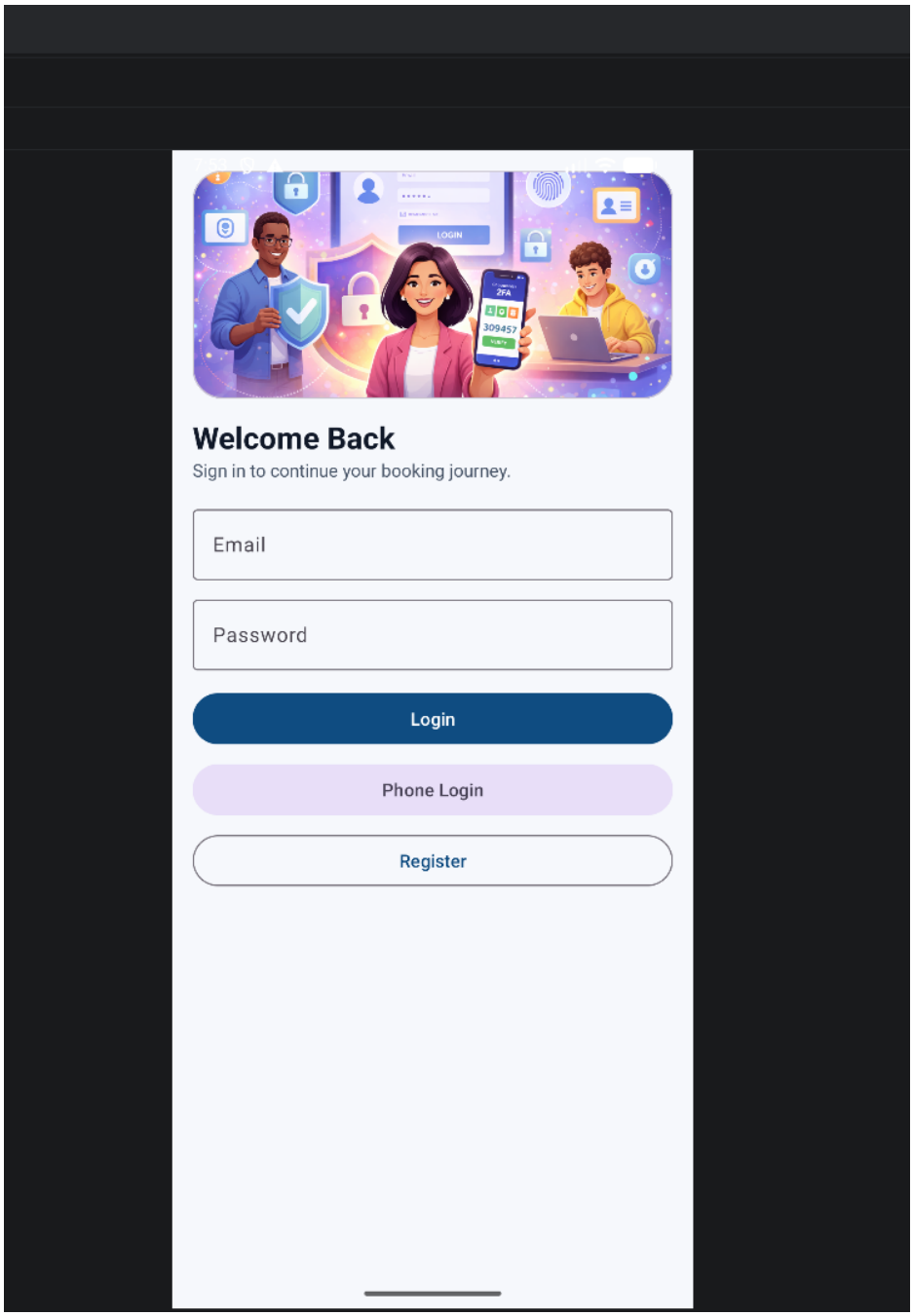

Figure 4: Login screen with email/password and phone OTP entry points.

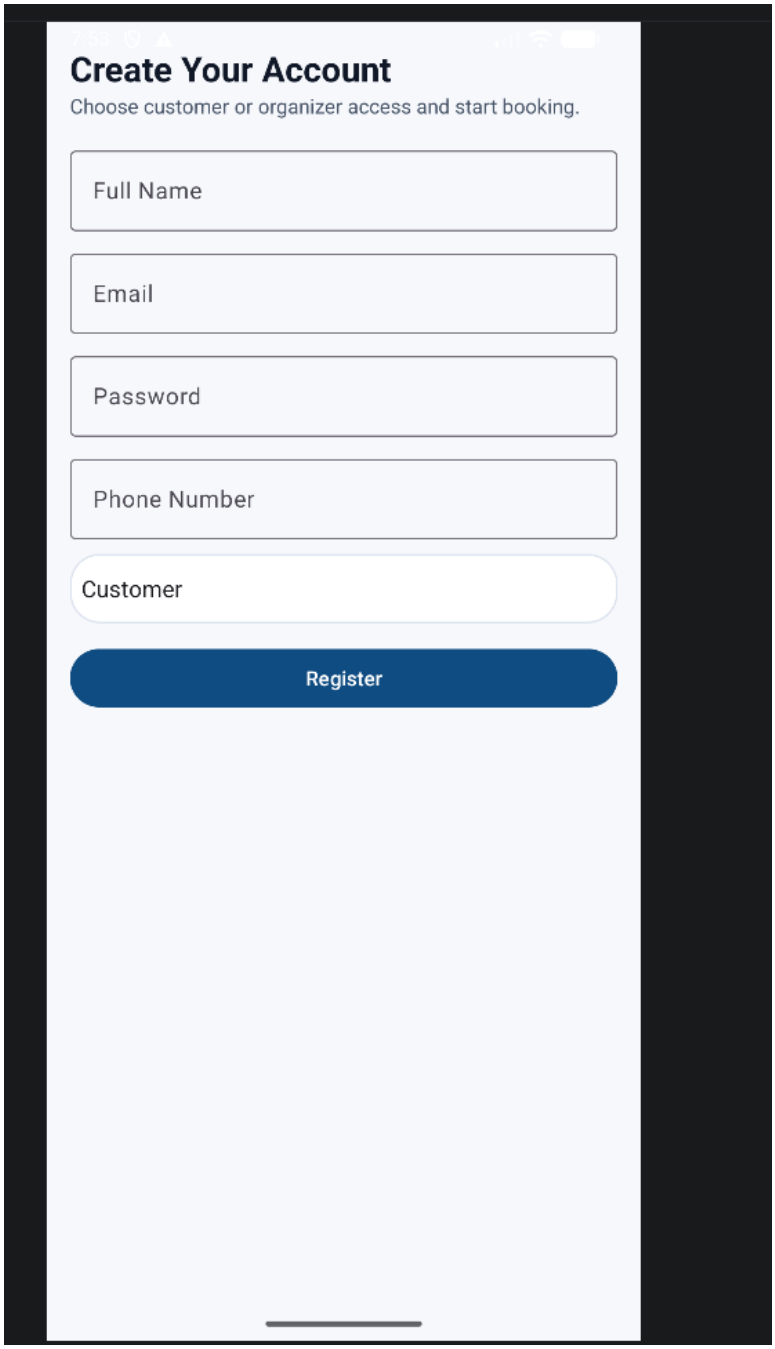

Figure 5: Registration screen for account creation.

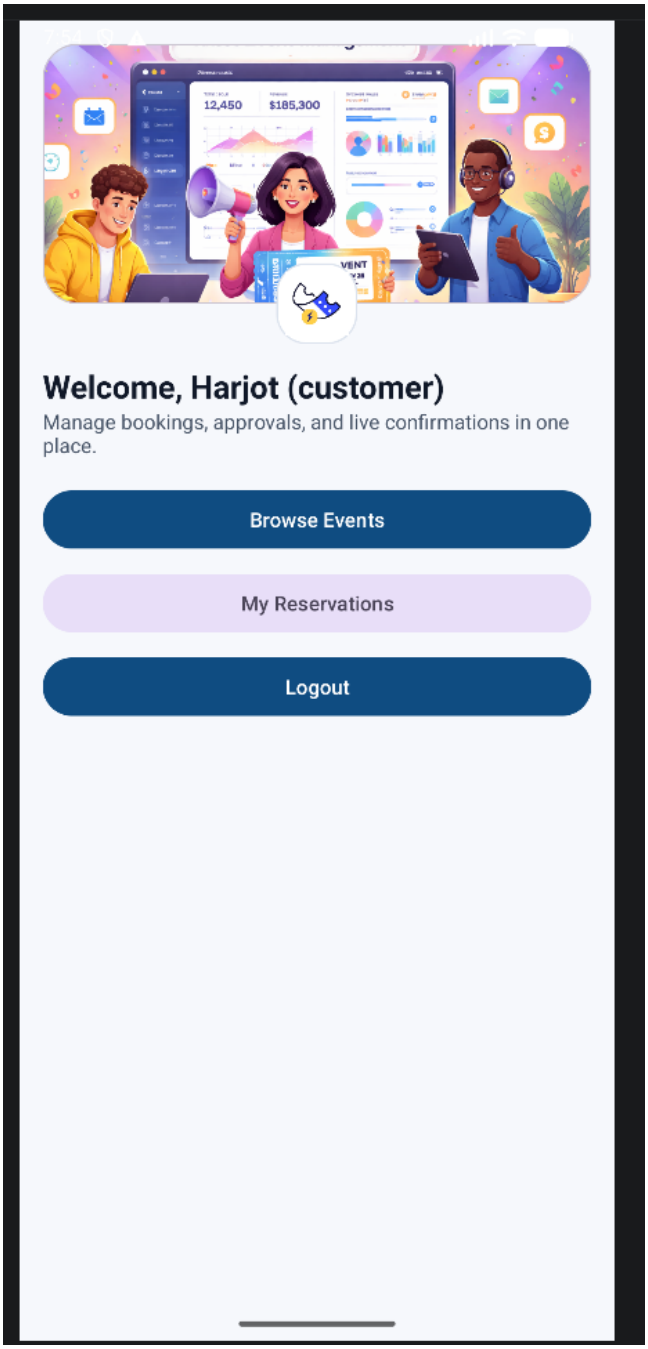

Figure 6: Customer dashboard after authentication.

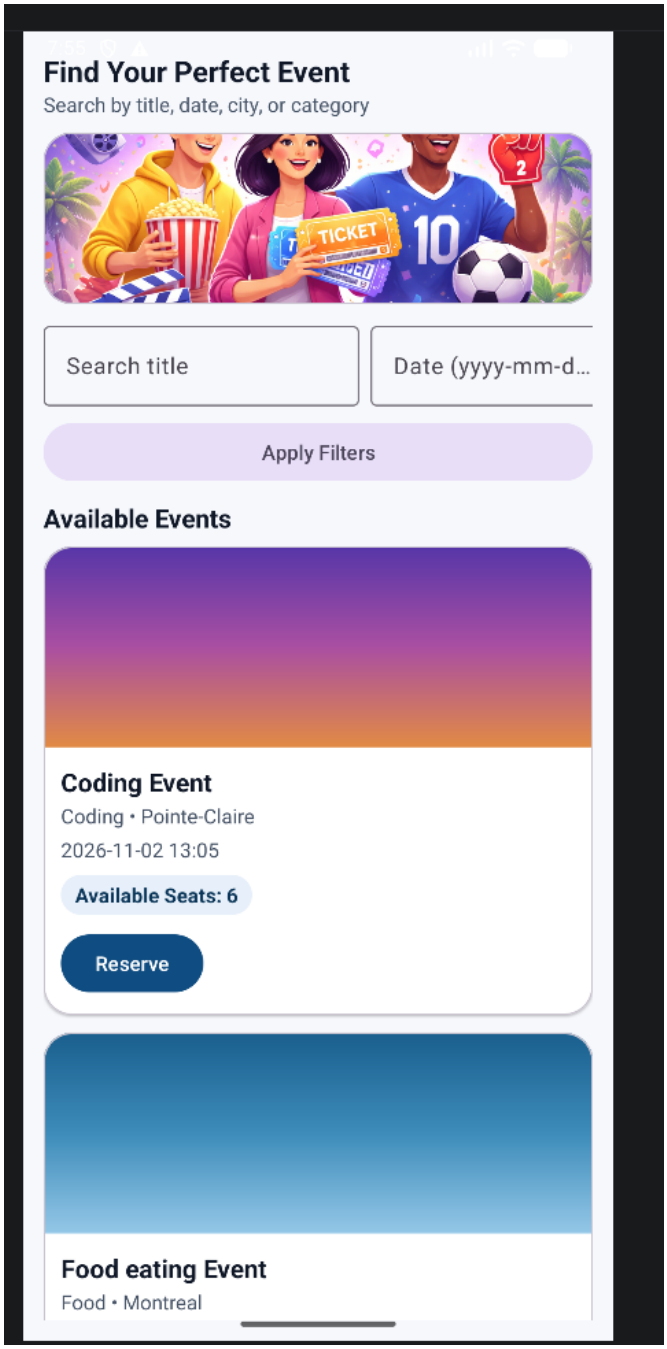

Figure 7: Customer event browsing and discovery view.

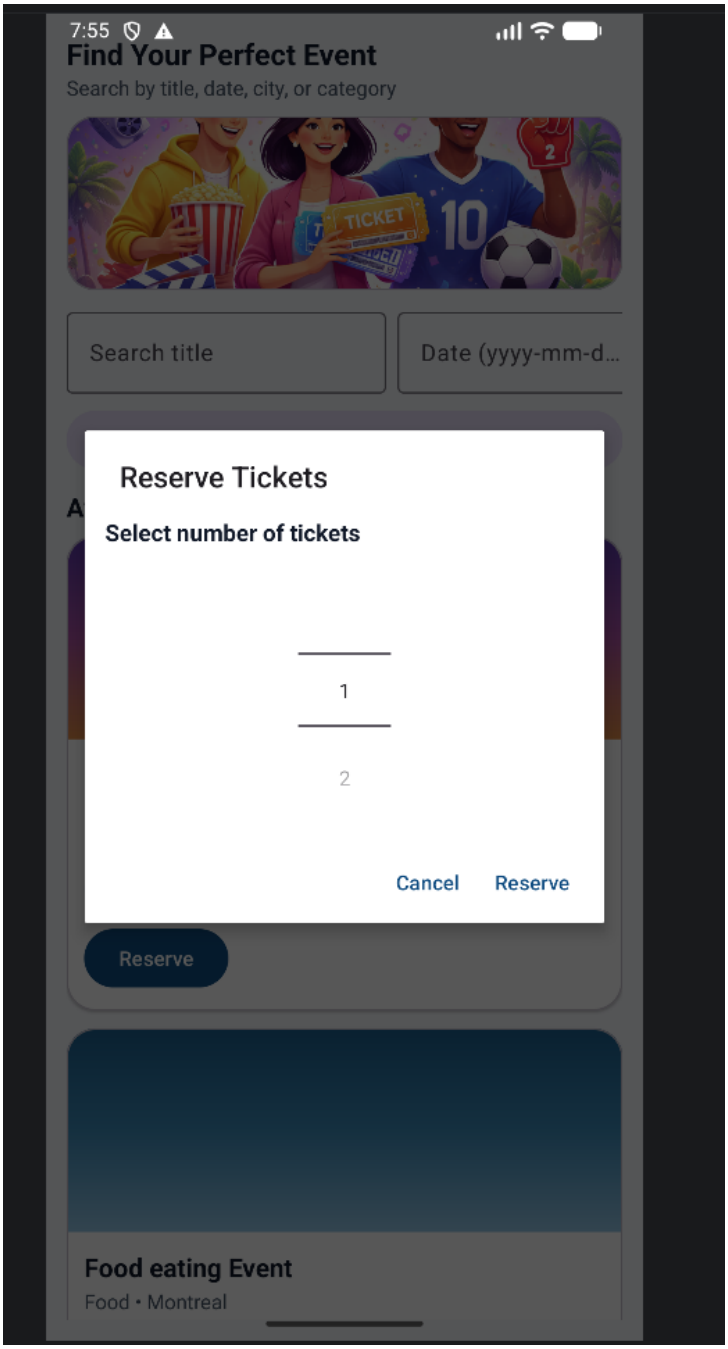

Figure 8: Ticket quantity input using the number-picker reservation dialog.

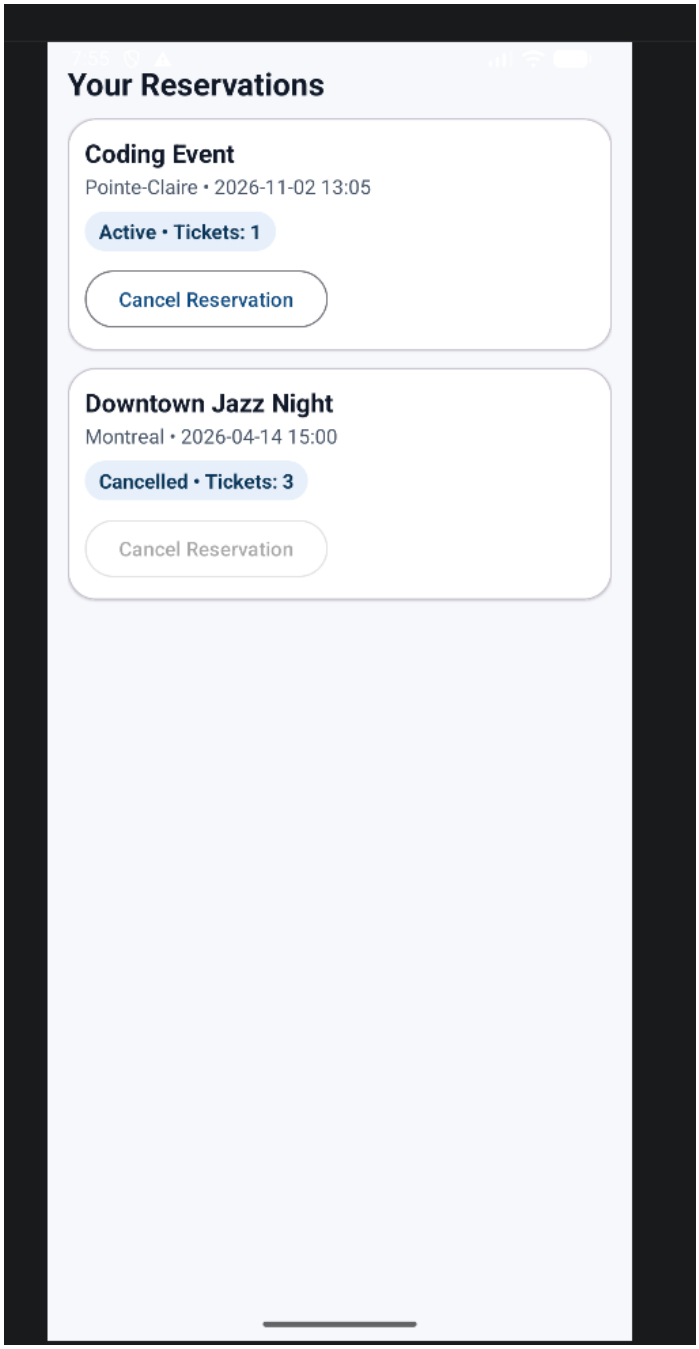

Figure 9: Customer reservation history and cancellation interface.

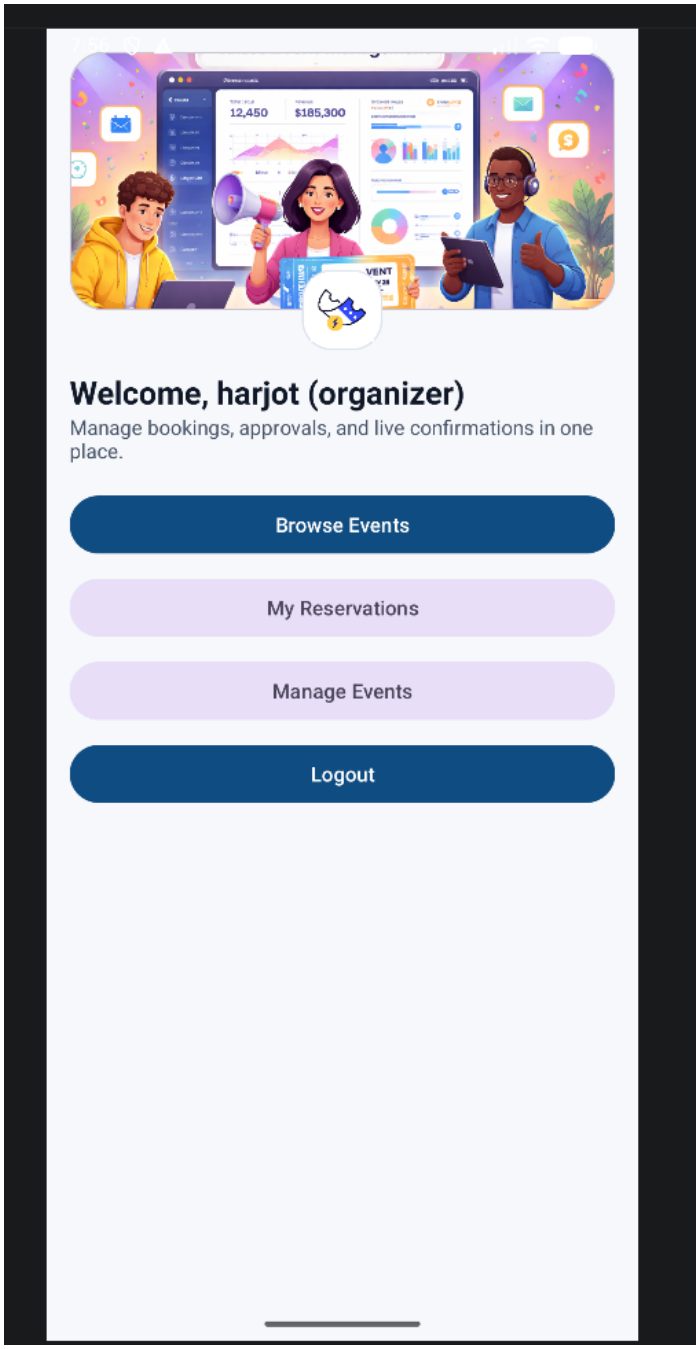

Figure 10: Organizer dashboard after login.

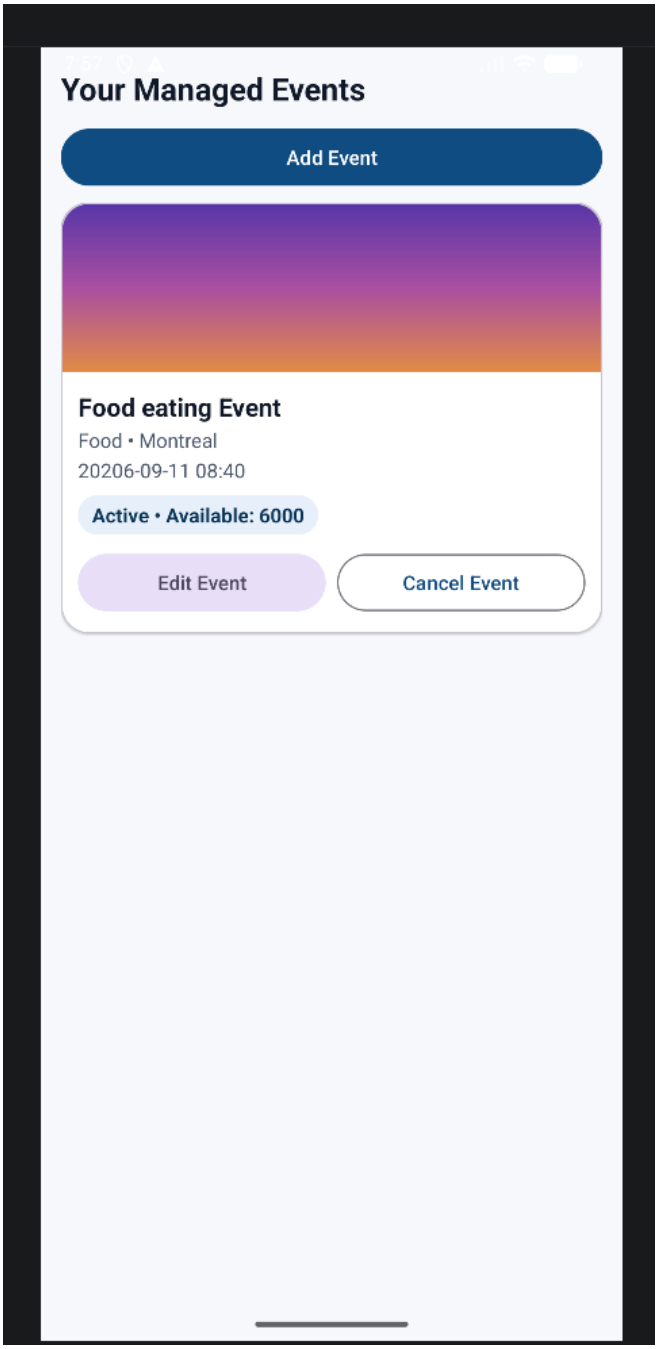

Figure 11: Organizer event management list.

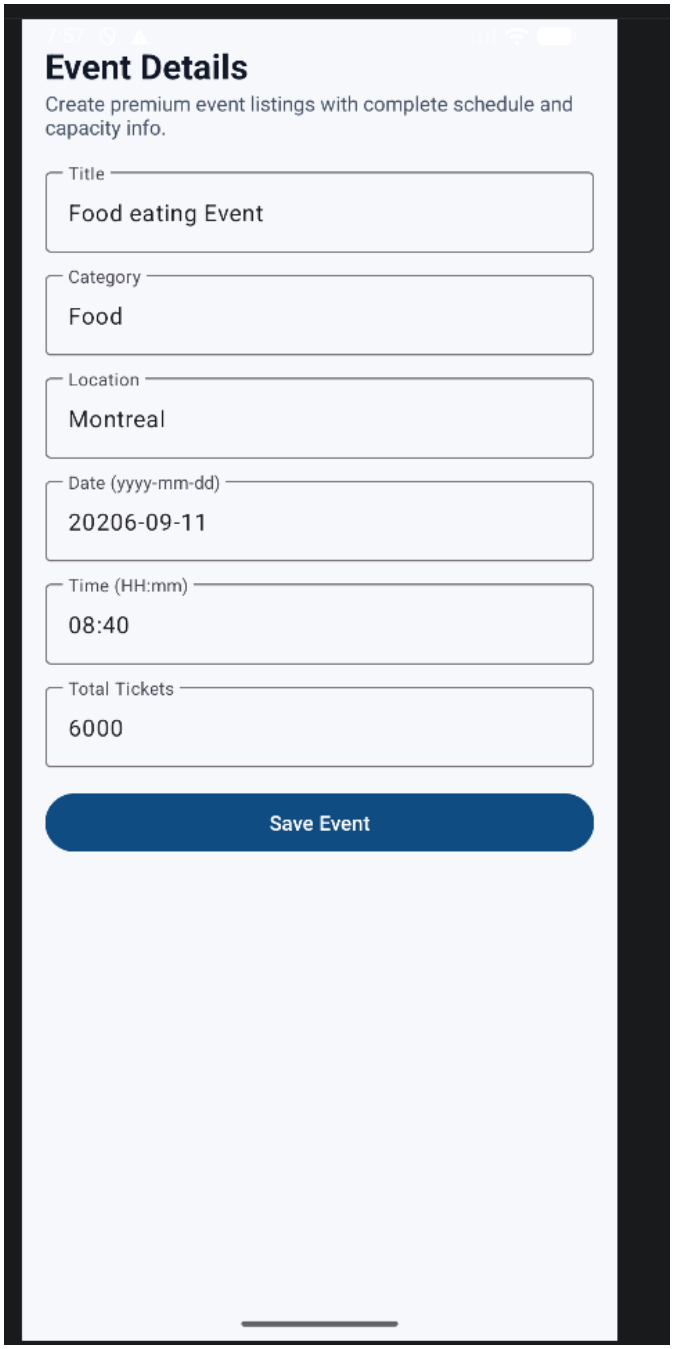

Figure 12: Organizer event editing form.

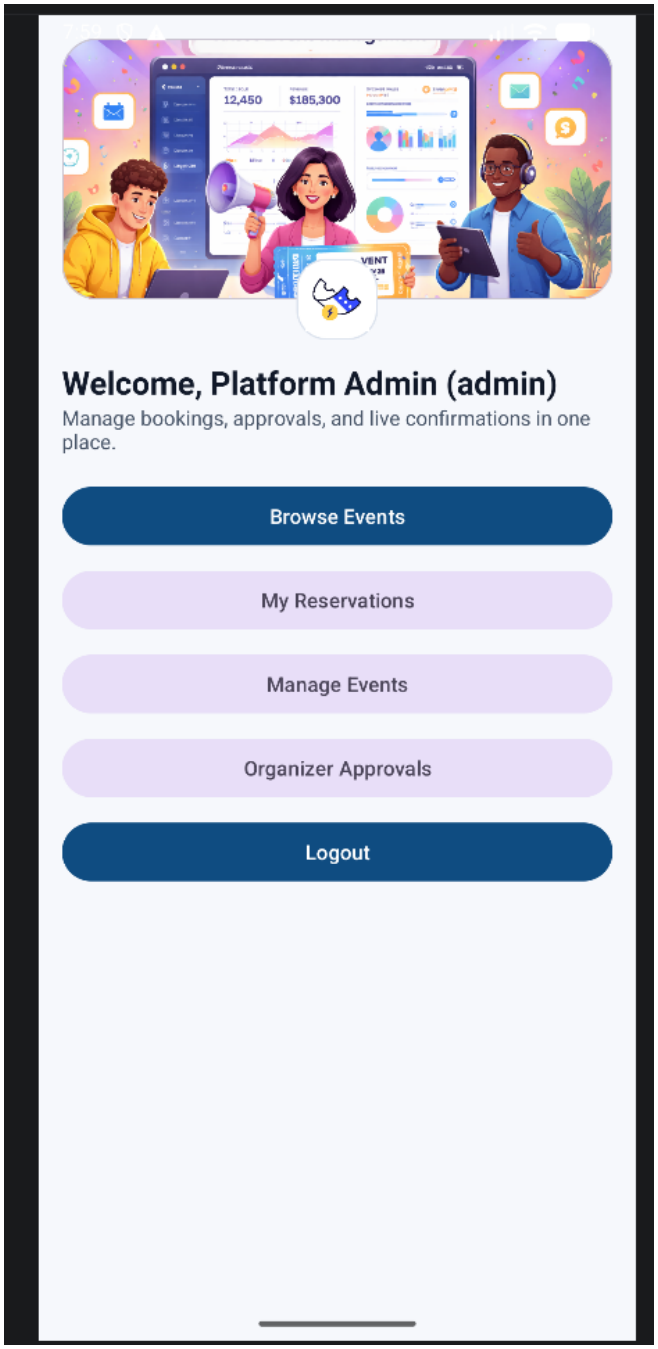

Figure 13: Admin dashboard.

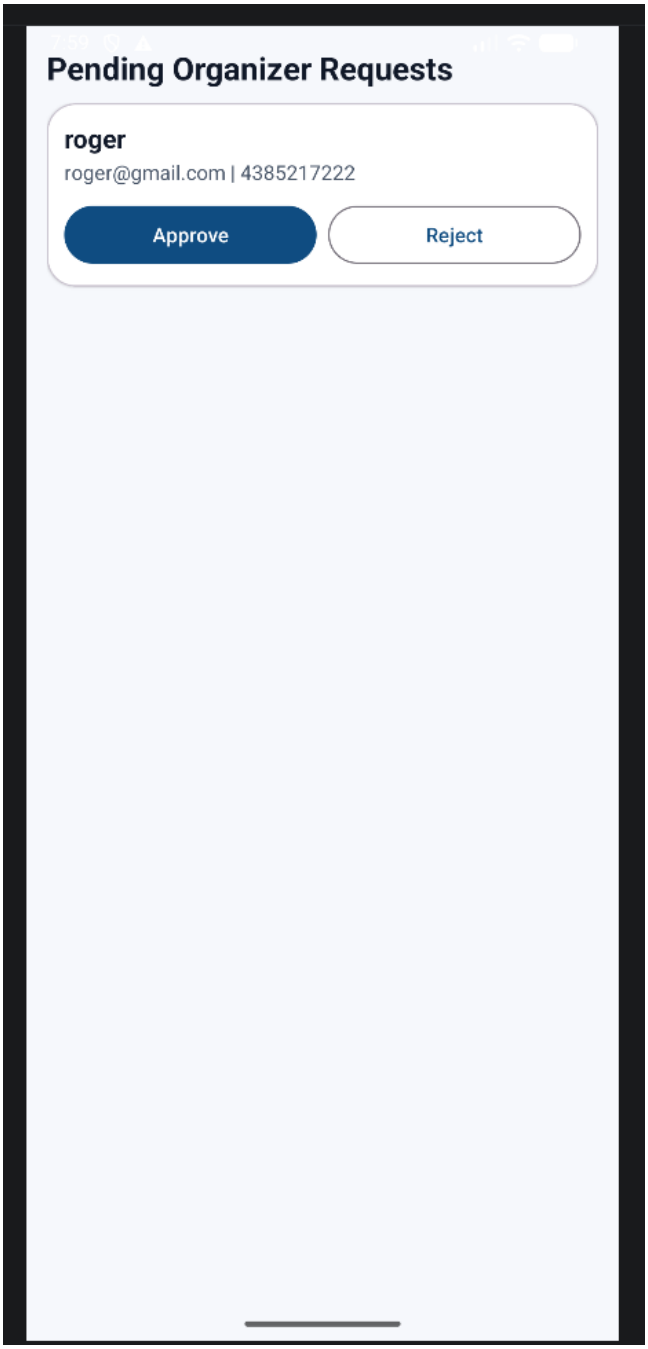

Figure 14: Admin approval queue for pending organizer requests.

8.2 Testing and Notification Evidence

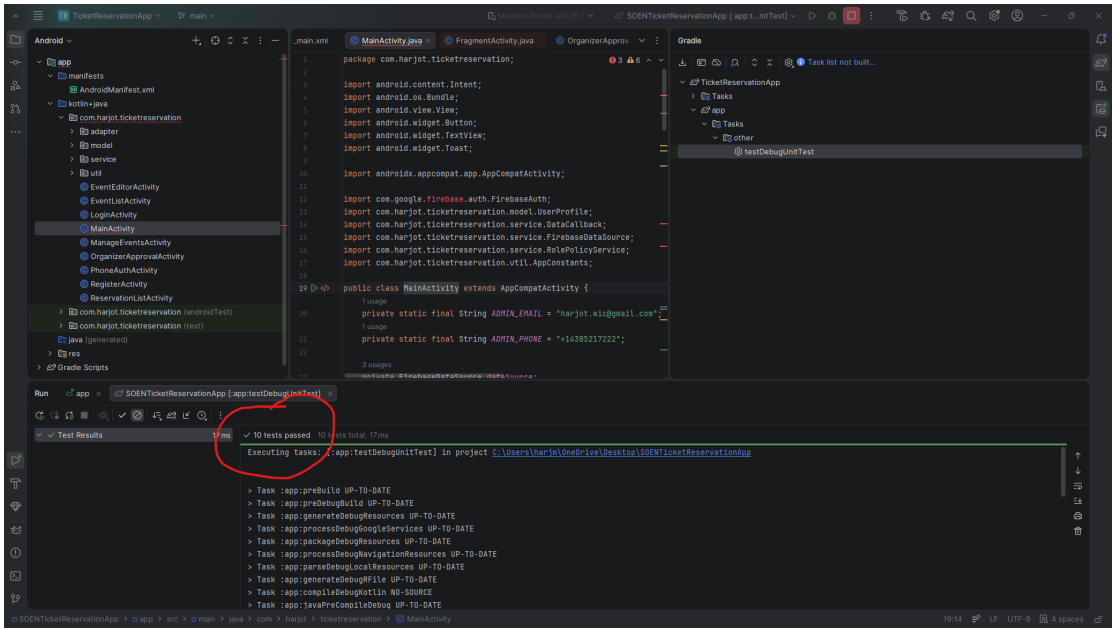

Figure 15: Unit and component testing execution output (`testDebugUnitTest`) showing a successful run.

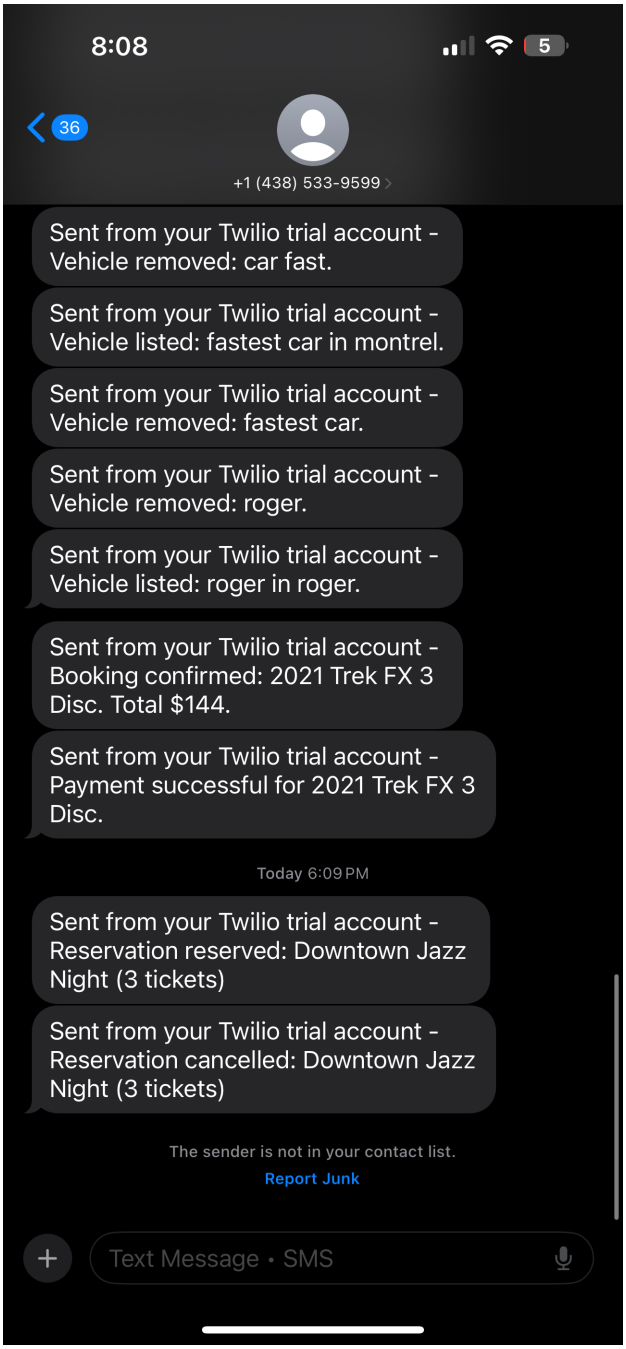

Figure 16: SMS notification evidence captured during system validation.

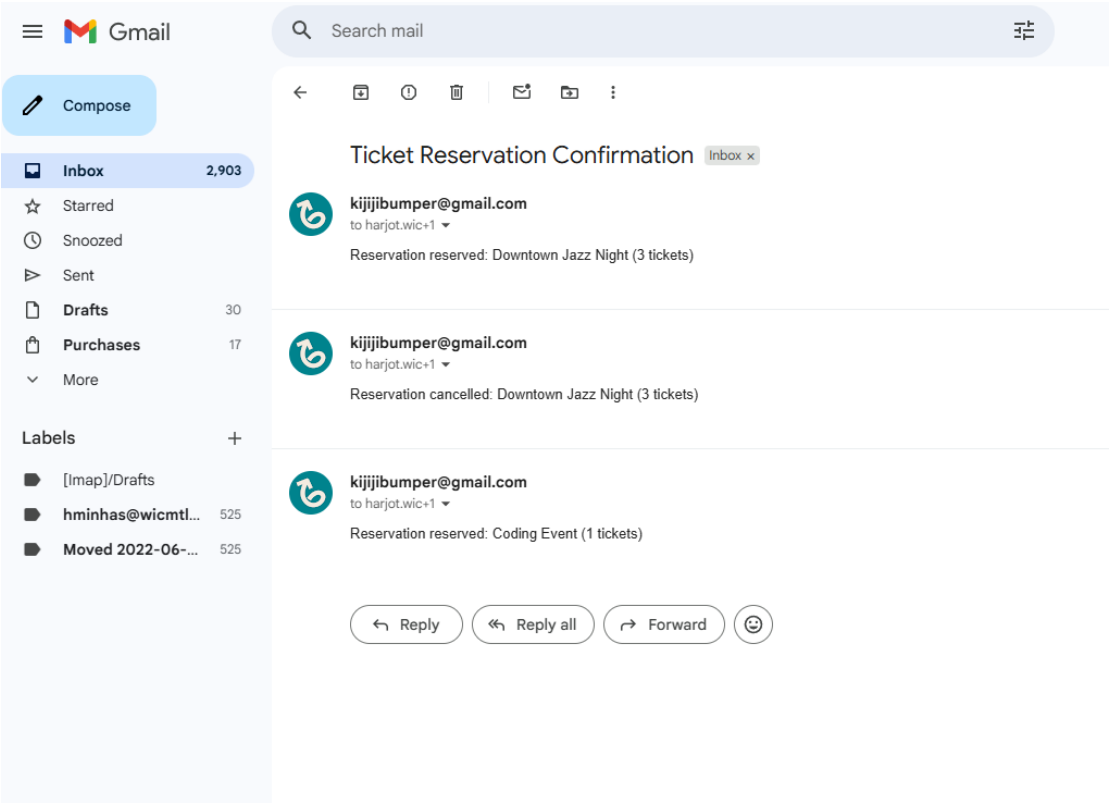

Figure 17: Email notification evidence captured during system validation.

8.3 Additional Evidence Files

The following image assets are included in `docs/images` and available as supporting evidence:

- `usecase-diagram.png`
- `class-diagram.png`
- `sequence-diagram.png`
- `login-page.png`
- `register-page.png`
- `customer-welcome-page.png`
- `customer-browse-events-page.png`
- `reserve-ticket-number-picker.png`
- `customer-my-reservations-page.png`
- `organizer-welcome-page.png`
- `organizer-manage-events-page.png`
- `organizer-edit-event-details.png`
- `admin-welcome-page.png`
- `admin-pending-organizer-requests.png`
- `test-unit-pass.png`
- `sms-notifications.png`

- `gmail-smtp-notification.png`

9 Conclusion

The project objectives were achieved with a complete cloud-based ticket reservation system that satisfies the specified functional and non-functional requirements. The final system includes role-based workflows, secure backend integration, transactional reservation consistency, and test-backed quality evidence. Continuous integration and comprehensive documentation support maintainability and reproducibility.

Appendix A: GitHub and Build Information

- Public repository: https://github.com/M1nha5/soen345-ticket-reservation
- Main Android package: `com.harjot.ticketreservation`
- Firebase project id: `soen345project-44582`
- Release artifact path: `app/build/outputs/apk/release/app-release.apk`